
This is a reduced pdf of an assignment set in previous years. It has been edited for clarity so you can see the initial problem students had to answer. This is not an assignment you have to hand in in any way. Note that the percentages will not add up to 100% as a theory section has been removed.

1 Introduction

`file(1)` is a classical Unix-like command-line utility for guessing the type of a file. In a Unix-like operating system, it is often the contents, or the metadata of a file, not a filename extension, that determines the “type” of a file. Standard `file(1)` uses clever heuristics to guess the file type. In A0, we will implement a basic variant of this tool in C.

We will only discern between the following small subset of the many possible file encodings.

empty: An empty file has 0 bytes.

ASCII text: An ASCII text file only contains characters from a particular subset of the ASCII-table.

ISO-8859-1 text: An ISO-8859-1 text file, also known as `latin1`, originating in 1987. It uses 8 bits instead of 7, and encodes most Latin alphabets, including Faroese, Icelandic, and Danish.

UTF-8 text: There are more Unicode (universal encoding) standards, but here we will only look at UTF-8 text. These standards aim to encompass the characters required to represent all the world’s languages, and are variable-width encodings (i.e., the number of bytes per character depends on the character). UTF-8 is today the predominant universal encoding. The later UTF-16 (which we do not work with) is regarded as an alternative, not an improvement over UTF-8.

data: Everything else is a data file.

2 API (10%)

Write a self-contained (one-file) C program, `file.c`, which when compiled to an executable file has the following API:

- 2.1. Let your `file` accept exactly one argument. If the given argument is a path to a file that exists, and the type of that file can be determined, write one line of text to `stdout`, and let `file` exit with the exit code 0 (`EXIT_SUCCESS`).

The line of text must consist of the given path, followed by a colon, space, and a guess of the types described above.

For instance, if there exists an ASCII text file called `ascii` in the same directory as your file binary:

```
$ ./file ascii
ascii: ASCII text
$ echo $?
0
```

The shell variable `$?` refers to the exit code of the previous command.

Similarly, if there exist corresponding files `data` and `empty`:

```
$ ./file datafile
datafile: data
$ ./file emptyfile
emptyfile: empty
```

Note: So far, the behaviour is exactly identical to `file(1)`. However, depending on your choice of characters, standard `file(1)` may report a *superstring*¹ of what your file reports. For instance, if there are no line-break characters (`\n`), `file(1)` reports:

```
$ printf "Hello, World!" > ascii
$ file ascii
ascii: ASCII text, with no line terminators
$ ./file ascii
ascii: ASCII text
```

More formally, your file must report a non-empty *substring* of the type that `file(1)` reports. Note that the above example uses the `printf(1)` shell command and not the `printf` C function.

- 2.2. If instead `file` is called with *no arguments*, it prints the usage message “Usage: file path” to `stderr`, and exits with `EXIT_FAILURE`:

```
$ ./file
Usage: file path
$ echo $?
1
```

Note: The usage string printed for `file(1)` is far more complicated, not least because it supports far more file types than we can hope to cover. `file(1)` however, also writes a usage message (starting with the word “Usage:”) to `stderr`, and exits with the exit code 1.

- 2.3. If we provide a path to a non-existing file, or some other I/O error occurs (e.g., someone rips out the USB-stick the file is on), behave as follows:

¹Note that a *superstring* denote to a string that contains the related string but may also contain more. On the other side, a *substring* can contain a smaller part of the related string.

```
$ ./file vulapyk
vulapyk: cannot determine (No such file or directory)
$ ./file hemmelig_fil
hemmelig_fil: cannot determine (Permission denied)
```

In both cases, file must still print to stdout and exit with EXIT_SUCCESS.

To achieve the above behaviour, we provide the following function:

```
// Assumes: errnum is a valid error number
int print_error(char *path, int errnum) {
    return fprintf(stdout, "%s: cannot determine (%s)\n",
        path, strerror(errnum));
}
```

This utilizes the standard strerror function to print a standard string for a corresponding error number, as set by the standard I/O functions.

In particular, if an error occurs during fopen(3), fseek(3), fread(3), or fclose(3), the standard C library will set the so-called errno variable. This variable becomes available if you include the errno.h library. The string.h library then provides the function strerror(3), which returns a string representation for an otherwise integer error code.

Note: file(1) behaves in a similar way: it exits with 0, but writes rather different messages to stdout. file(1) does not regard invalid paths and I/O errors as errors on its part. This is a questionable design decision.

3 File Types (15%)

Your file should recognise the following file types:

empty

This file type must be reported if the file contains no bytes.

ASCII text

This file type must be reported if all bytes belong to the following set:

$$\{0x07, 0x08, \dots, 0x0D\} \cup \{0x1B\} \cup \{0x20, 0x21, \dots, 0x7E\}$$

ISO-8859 text

This file type should be reported if the file composed of ISO-8859-1-like bytes. These include all ASCII-like bytes (see above), and also decimal values 160–255. The decimal values 128–159 are not part of ISO-8859-1, and their appearance might indicate that the file really is a UTF-8-encoded text file. (Why?)

UTF-8 Unicode text

This file type should be reported if the file is composed of UTF-8-like characters. UTF-8 is a variable-length encoding where each subsequent

byte of a character begins with a designated bit-sequence. The following table summarises the encoding:

Number of Bytes	Byte 1	Byte 2	Byte 3	Byte 4
1	0xxxxxxx			
2	110xxxxx	10xxxxxx		
3	1110xxxx	10xxxxxx	10xxxxxx	
4	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

data

This file type must be reported in case no other type matches.

4 Testing (25%)

The assignment has been carefully designed so that you can test your implementation by comparing its observable behaviour with that of the standard `file(1)` utility.

To this end, we hand out a shell-script `test.sh`. Run it as follows:

```
$ bash test.sh
```

This script will create a directory `test_files`, and fill it with sample files.

The script will then compare the expected output (generated by `file(1)`, filtered to remove anything following ASCII text), and the actual output (generated by your `file` utility). If the files differ, the script will fail.

The comparison is done using the standard `diff(1)` utility. You should be familiar with `diff`'s from Software Development. See also Appendix ??.

Your task: make it generate more interesting sample files, such that you get a covering unit testing. That is:

- Generate at least a dozen relevant files for testing of each relevant text encoding.
- Generate at least a dozen files for testing data.
- Is the empty file type already tested?

Consider and document how well your test covers the input problem. What is tested well and what is hard to test?

5 Report (40%)

Alongside your solution (implemented code and tests), you should submit a short report; must not exceed 5 pages of textual content excluding the theoretical part².

The report *must*:

- Have a title that states the assignment solved on the first page

²These 5 pages is a strict upper limit. Quality over size; less can easily be a very good report.

- Clearly state the names and KUIDs of all group members on the first page

The report should:

- 5.1. Shortly introduce the problem.
- 5.2. Discuss your approach to it and your design decisions.
- 5.3. Disambiguate any ambiguities you might have found in the assignment.
- 5.4. Present the non-trivial parts of your implementation.
- 5.5. Describe how to compile your code and run your tests to reproduce your test results.
- 5.6. Discuss the coverage of your test and what the outcome were.
- 5.7. Conclude on the outcome of the entire solution.
- 5.8. Answers to the theoretical questions

See the given report example for inspiration and details.